

Task 1:

```
In [ ]: import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("../images/cameraman.tif", cv2.IMREAD_GRAYSCALE)

if img is None:
    raise FileNotFoundError("Image could not be loaded.")
```

```
In [ ]: height, width = img.shape

resized = cv2.resize(
    img,
    (width * 2, height * 2),
    interpolation=cv2.INTER_LINEAR
)

plt.imshow(resized, cmap="gray", vmin=0, vmax=255)
plt.title("2x Enlarged Image")
plt.axis("off")
plt.show()
```

2x Enlarged Image



```
In [ ]: height, width = img.shape

        center = (width / 2, height / 2)

        rotation_matrix = cv2.getRotationMatrix2D(
            center,
            120,
            1.0
        )

        rotated = cv2.warpAffine(
            img,
            rotation_matrix,
            (width, height)
        )

        plt.imshow(rotated, cmap="gray", vmin=0, vmax=255)
        plt.title("Rotated 120 Degrees")
        plt.axis("off")
        plt.show()
```

Rotated 120 Degrees



```
In [ ]: height, width = img.shape

shear_factor = 0.5

shear_matrix = np.float32([
    [1, shear_factor, 0],
    [0, 1, 0]
])

sheared = cv2.warpAffine(
    img,
    shear_matrix,
    (int(width + shear_factor * height), height)
)

plt.imshow(sheared, cmap="gray", vmin=0, vmax=255)
plt.title("Sheared Image")
plt.axis("off")
plt.show()
```

Sheared Image



Task 2:

```
In [ ]: negative = 255 - img

plt.imshow(negative, cmap="gray", vmin=0, vmax=255)
plt.title("Negative Image")
plt.axis("off")
plt.show()
```

Negative Image



```
In [ ]: c = 255 / np.log(256)

log_image = c * np.log(1 + img.astype(np.float32))
log_image = np.uint8(log_image)

plt.imshow(log_image, cmap="gray", vmin=0, vmax=255)
plt.title("Log Transformation")
plt.axis("off")
plt.show()
```

Log Transformation



```
In [ ]: gamma = 0.5

normalized = img / 255.0

power_image = np.power(normalized, gamma)

power_image = np.uint8(power_image * 255)

plt.imshow(power_image, cmap="gray", vmin=0, vmax=255)
plt.title("Power-Law Transformation (Gamma = 0.5)")
plt.axis("off")
plt.show()
```

Power-Law Transformation (Gamma = 0.5)

